Air University

Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: April 02, 2026

Lab # 07

1. Code

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <cstring>
#include <random>
#include <algorithm>
using namespace std;

//  AES CONSTANT
static const uint8_t SBOX[256] = {
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab,
0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72,
0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31,
0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2,
0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f,
0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58,
0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f,
0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3,
0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19,
0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b,
0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4,
0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae,
0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b,
0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d,
0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28,
0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb,
0x16};

static const uint8_t INV_SBOX[256] = {
    0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e, 0x81, 0xf3, 0xd7,
0xfb,
```

```
    0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43, 0x44, 0xc4, 0xde, 0xe9,
0xcb,
    0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42, 0xfa, 0xc3,
0x4e,
    0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b, 0xd1,
0x25,
    0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c, 0xcc, 0x5d, 0x65, 0xb6,
0x92,
    0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46, 0x57, 0xa7, 0x8d, 0x9d,
0x84,
    0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4, 0x58, 0x05, 0xb8, 0xb3, 0x45,
0x06,
    0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13, 0x8a,
0x6b,
    0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf, 0xce, 0xf0, 0xb4, 0xe6,
0x73,
    0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37, 0xe8, 0x1c, 0x75, 0xdf,
0x6e,
    0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7, 0x62, 0x0e, 0xaa, 0x18, 0xbe,
0x1b,
    0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78, 0xcd, 0x5a,
0xf4,
    0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xec,
0x5f,
    0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a, 0x9f, 0x93, 0xc9, 0x9c,
0xef,
    0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb, 0xbb, 0x3c, 0x83, 0x53, 0x99,
0x61,
    0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0c,
0x7d};

static const uint8_t RCON[11] = {
    0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b, 0x36};

//  GALOIS FIELD MULTIPLICATION
uint8_t gf_mul(uint8_t a, uint8_t b)
{
    uint8_t result = 0;
    while (b)
    {
        if (b & 1)
            result ^= a;
        bool hi = a & 0x80;
        a <<= 1;
        if (hi)
            a ^= 0x1B;
        b >>= 1;
    }
    return result;
}

//  AES STATE OPERATIONS
```

```
void subBytes(uint8_t s[4][4])
{
    for (int r = 0; r < 4; r++)
        for (int c = 0; c < 4; c++)
            s[r][c] = SBOX[s[r][c]];
}
void invSubBytes(uint8_t s[4][4])
{
    for (int r = 0; r < 4; r++)
        for (int c = 0; c < 4; c++)
            s[r][c] = INV_SBOX[s[r][c]];
}

void shiftRows(uint8_t s[4][4])
{
    for (int r = 1; r < 4; r++)
    {
        uint8_t t[4];
        for (int c = 0; c < 4; c++)
            t[c] = s[r][(c + r) % 4];
        for (int c = 0; c < 4; c++)
            s[r][c] = t[c];
    }
}
void invShiftRows(uint8_t s[4][4])
{
    for (int r = 1; r < 4; r++)
    {
        uint8_t t[4];
        for (int c = 0; c < 4; c++)
            t[c] = s[r][(c - r + 4) % 4];
        for (int c = 0; c < 4; c++)
            s[r][c] = t[c];
    }
}

void mixColumns(uint8_t s[4][4])
{
    for (int c = 0; c < 4; c++)
    {
        uint8_t s0 = s[0][c], s1 = s[1][c], s2 = s[2][c], s3 = s[3][c];
        s[0][c] = gf_mul(2, s0) ^ gf_mul(3, s1) ^ s2 ^ s3;
        s[1][c] = s0 ^ gf_mul(2, s1) ^ gf_mul(3, s2) ^ s3;
        s[2][c] = s0 ^ s1 ^ gf_mul(2, s2) ^ gf_mul(3, s3);
        s[3][c] = gf_mul(3, s0) ^ s1 ^ s2 ^ gf_mul(2, s3);
    }
}
void invMixColumns(uint8_t s[4][4])
{
    for (int c = 0; c < 4; c++)
    {
        uint8_t s0 = s[0][c], s1 = s[1][c], s2 = s[2][c], s3 = s[3][c];
```

```
        s[0][c] = gf_mul(0x0e, s0) ^ gf_mul(0x0b, s1) ^ gf_mul(0x0d, s2) ^ gf_mul(0x09, s3);
        s[1][c] = gf_mul(0x09, s0) ^ gf_mul(0x0e, s1) ^ gf_mul(0x0b, s2) ^ gf_mul(0x0d, s3);
        s[2][c] = gf_mul(0x0d, s0) ^ gf_mul(0x09, s1) ^ gf_mul(0x0e, s2) ^ gf_mul(0x0b, s3);
        s[3][c] = gf_mul(0x0b, s0) ^ gf_mul(0x0d, s1) ^ gf_mul(0x09, s2) ^ gf_mul(0x0e, s3);
    }
}

void addRoundKey(uint8_t s[4][4], const vector<uint32_t> &w, int round)
{
    for (int c = 0; c < 4; c++)
    {
        uint32_t word = w[round * 4 + c];
        s[0][c] ^= (word >> 24) & 0xFF;
        s[1][c] ^= (word >> 16) & 0xFF;
        s[2][c] ^= (word >> 8) & 0xFF;
        s[3][c] ^= (word) & 0xFF;
    }
}

//  KEY EXPANSION
vector<uint32_t> keyExpansion(const vector<uint8_t> &key)
{
    int Nk = key.size() / 4, Nr = Nk + 6, total = 4 * (Nr + 1);
    vector<uint32_t> w(total);
    for (int i = 0; i < Nk; i++)
        w[i] = ((uint32_t)key[4 * i] << 24) | ((uint32_t)key[4 * i + 1] << 16) |
((uint32_t)key[4 * i + 2] << 8) | ((uint32_t)key[4 * i + 3]);
    for (int i = Nk; i < total; i++)
    {
        uint32_t tmp = w[i - 1];
        if (i % Nk == 0)
        {
            tmp = ((tmp << 8) | (tmp >> 24));
            tmp = ((uint32_t)SBOX[(tmp >> 24) & 0xFF] << 24) | ((uint32_t)SBOX[(tmp >> 16) &
0xFF] << 16) | ((uint32_t)SBOX[(tmp >> 8) & 0xFF] << 8) | ((uint32_t)SBOX[(tmp) & 0xFF]);
            tmp ^= ((uint32_t)RCON[i / Nk] << 24);
        }
        else if (Nk > 6 && i % Nk == 4)
        {
            tmp = ((uint32_t)SBOX[(tmp >> 24) & 0xFF] << 24) | ((uint32_t)SBOX[(tmp >> 16) &
0xFF] << 16) | ((uint32_t)SBOX[(tmp >> 8) & 0xFF] << 8) | ((uint32_t)SBOX[(tmp) & 0xFF]);
        }
        w[i] = w[i - Nk] ^ tmp;
    }
    return w;
}

//  BLOCK ENCRYPT / DECRYPT
vector<uint8_t> aesEncryptBlock(const vector<uint8_t> &block,
                                const vector<uint32_t> &w, int Nr)
{
    uint8_t s[4][4];
```

```
    for (int r = 0; r < 4; r++)
        for (int c = 0; c < 4; c++)
            s[r][c] = block[r + 4 * c];
    addRoundKey(s, w, 0);
    for (int round = 1; round <= Nr; round++)
    {
        subBytes(s);
        shiftRows(s);
        if (round != Nr)
            mixColumns(s);
        addRoundKey(s, w, round);
    }
    vector<uint8_t> out(16);
    for (int r = 0; r < 4; r++)
        for (int c = 0; c < 4; c++)
            out[r + 4 * c] = s[r][c];
    return out;
}

vector<uint8_t> aesDecryptBlock(const vector<uint8_t> &block,
                                const vector<uint32_t> &w, int Nr)
{
    uint8_t s[4][4];
    for (int r = 0; r < 4; r++)
        for (int c = 0; c < 4; c++)
            s[r][c] = block[r + 4 * c];
    addRoundKey(s, w, Nr);
    for (int round = Nr - 1; round >= 0; round--)
    {
        invShiftRows(s);
        invSubBytes(s);
        addRoundKey(s, w, round);
        if (round != 0)
            invMixColumns(s);
    }
    vector<uint8_t> out(16);
    for (int r = 0; r < 4; r++)
        for (int c = 0; c < 4; c++)
            out[r + 4 * c] = s[r][c];
    return out;
}

//  PKCS#7 PADDING
vector<uint8_t> addPadding(const vector<uint8_t> &data)
{
    int pad = 16 - (data.size() % 16);
    vector<uint8_t> p = data;
    p.insert(p.end(), pad, (uint8_t)pad);
    return p;
}
vector<uint8_t> removePadding(const vector<uint8_t> &data)
{
```

```
    if (data.empty())
        return data;
    uint8_t pad = data.back();
    if (pad == 0 || pad > 16)
        return data;
    return vector<uint8_t>(data.begin(), data.end() - pad);
}

//  ECB MODE
vector<uint8_t> aesECBEncrypt(const vector<uint8_t> &pt,
                              const vector<uint8_t> &key)
{
    int Nr = key.size() / 4 + 6;
    auto w = keyExpansion(key);
    auto padded = addPadding(pt);
    vector<uint8_t> ct;
    for (size_t i = 0; i < padded.size(); i += 16)
    {
        auto enc = aesEncryptBlock({padded.begin() + i, padded.begin() + i + 16}, w, Nr);
        ct.insert(ct.end(), enc.begin(), enc.end());
    }
    return ct;
}
vector<uint8_t> aesECBDecrypt(const vector<uint8_t> &ct,
                              const vector<uint8_t> &key)
{
    int Nr = key.size() / 4 + 6;
    auto w = keyExpansion(key);
    vector<uint8_t> pt;
    for (size_t i = 0; i < ct.size(); i += 16)
    {
        auto dec = aesDecryptBlock({ct.begin() + i, ct.begin() + i + 16}, w, Nr);
        pt.insert(pt.end(), dec.begin(), dec.end());
    }
    return removePadding(pt);
}

//  CBC MODE
vector<uint8_t> aesCBCEncrypt(const vector<uint8_t> &pt,
                              const vector<uint8_t> &key,
                              const vector<uint8_t> &iv)
{
    int Nr = key.size() / 4 + 6;
    auto w = keyExpansion(key);
    auto padded = addPadding(pt);
    vector<uint8_t> ct, prev = iv;
    for (size_t i = 0; i < padded.size(); i += 16)
    {
        vector<uint8_t> block(padded.begin() + i, padded.begin() + i + 16);
        for (int j = 0; j < 16; j++)
            block[j] ^= prev[j];
        auto enc = aesEncryptBlock(block, w, Nr);
```

```
        ct.insert(ct.end(), enc.begin(), enc.end());
        prev = enc;
    }
    return ct;
}
vector<uint8_t> aesCBCDecrypt(const vector<uint8_t> &ct,
                              const vector<uint8_t> &key,
                              const vector<uint8_t> &iv)
{
    int Nr = key.size() / 4 + 6;
    auto w = keyExpansion(key);
    vector<uint8_t> pt;
    vector<uint8_t> prev = iv;
    for (size_t i = 0; i < ct.size(); i += 16)
    {
        vector<uint8_t> block(ct.begin() + i, ct.begin() + i + 16);
        auto dec = aesDecryptBlock(block, w, Nr);
        for (int j = 0; j < 16; j++)
            dec[j] ^= prev[j];
        pt.insert(pt.end(), dec.begin(), dec.end());
        prev = block;
    }
    return removePadding(pt);
}

//  UTILITIES
vector<uint8_t> randomBytes(int n)
{
    static mt19937 rng(random_device{}());
    static uniform_int_distribution<int> dist(0, 255);
    vector<uint8_t> buf(n);
    for (auto &b : buf)
        b = dist(rng);
    return buf;
}

void printHex(const string &label, const vector<uint8_t> &data)
{
    cout << label;
    for (auto b : data)
        cout << hex << setw(2) << setfill('0') << (int)b;
    cout << dec << "\n";
}

vector<uint8_t> toBytes(const string &s)
{
    return vector<uint8_t>(s.begin(), s.end());
}
string toString(const vector<uint8_t> &v)
{
    return string(v.begin(), v.end());
}
```

```cpp
int countDiffBits(const vector<uint8_t> &a, const vector<uint8_t> &b)
{
    int bits = 0;
    for (size_t i = 0; i < min(a.size(), b.size()); i++)
    {
        uint8_t d = a[i] ^ b[i];
        while (d)
        {
            bits += d & 1;
            d >>= 1;
        }
    }
    return bits;
}

void printSeparator(const string &title)
{
    cout << "\n============================================\n";
    cout << "  " << title << "\n";
    cout << "============================================\n";
}

//  MAIN
int main()
{
    const string plaintext = "CRYPTOGRAPHY IS FUN";
    vector<uint8_t> pt = toBytes(plaintext);

    // ── TASK 1: AES-128 ECB ──────────────────────────
    printSeparator("TASK 1: AES-128 ECB Encryption & Decryption");

    vector<uint8_t> key128 = randomBytes(16);
    vector<uint8_t> ecb_ct = aesECBEncrypt(pt, key128);
    vector<uint8_t> ecb_dec = aesECBDecrypt(ecb_ct, key128);

    printHex("Key (128-bit)  : ", key128);
    cout << "Plaintext      : \"" << plaintext << "\"\n";
    cout << "Plaintext Len  : " << pt.size()
         << " bytes padded to "
         << ((pt.size() / 16) + 1) * 16 << " bytes (PKCS#7)\n";
    printHex("ECB Ciphertext : ", ecb_ct);
    cout << "ECB Decrypted  : \"" << toString(ecb_dec) << "\"\n";
    cout << "Verification   : "
         << (toString(ecb_dec) == plaintext ? "PASS [OK]" : "FAIL") << "\n";

    // ── TASK 2: AES-128 CBC vs ECB ──────────────────
    printSeparator("TASK 2: AES-128 CBC Mode & ECB vs CBC Comparison");

    vector<uint8_t> iv128 = randomBytes(16);
    vector<uint8_t> cbc_ct = aesCBCEncrypt(pt, key128, iv128);
    vector<uint8_t> cbc_dec = aesCBCDecrypt(cbc_ct, key128, iv128);
```

```
    printHex("IV  (128-bit)  : ", iv128);
    printHex("CBC Ciphertext : ", cbc_ct);
    cout << "CBC Decrypted  : \"" << toString(cbc_dec) << "\"\n";
    cout << "Verification   : "
         << (toString(cbc_dec) == plaintext ? "PASS [OK]" : "FAIL") << "\n";

    cout << "\n--- ECB vs CBC Ciphertext Comparison ---\n";
    printHex("ECB            : ", ecb_ct);
    printHex("CBC            : ", cbc_ct);
    cout << "Match          : "
         << (ecb_ct == cbc_ct ? "YES (unexpected!)" : "NO  (expected)") << "\n";

    // ── TASK 3: AES-192 & AES-256 ──────────────────────────
    printSeparator("TASK 3: Key Size Variation");

    // AES-192
    vector<uint8_t> key192 = randomBytes(24);
    vector<uint8_t> iv192 = randomBytes(16);
    vector<uint8_t> ct192_ecb = aesECBEncrypt(pt, key192);
    vector<uint8_t> ct192_cbc = aesCBCEncrypt(pt, key192, iv192);
    vector<uint8_t> dc192_ecb = aesECBDecrypt(ct192_ecb, key192);
    vector<uint8_t> dc192_cbc = aesCBCDecrypt(ct192_cbc, key192, iv192);

    cout << "\n[ AES-192 ]\n";
    printHex("Key (192-bit)  : ", key192);
    printHex("ECB Ciphertext : ", ct192_ecb);
    cout << "ECB Decrypted  : \"" << toString(dc192_ecb) << "\"\n";
    cout << "ECB Verify     : "
         << (toString(dc192_ecb) == plaintext ? "PASS [OK]" : "FAIL") << "\n";
    printHex("IV  (128-bit)  : ", iv192);
    printHex("CBC Ciphertext : ", ct192_cbc);
    cout << "CBC Decrypted  : \"" << toString(dc192_cbc) << "\"\n";
    cout << "CBC Verify     : "
         << (toString(dc192_cbc) == plaintext ? "PASS [OK]" : "FAIL") << "\n";

    // AES-256
    vector<uint8_t> key256 = randomBytes(32);
    vector<uint8_t> iv256 = randomBytes(16);
    vector<uint8_t> ct256_ecb = aesECBEncrypt(pt, key256);
    vector<uint8_t> ct256_cbc = aesCBCEncrypt(pt, key256, iv256);
    vector<uint8_t> dc256_ecb = aesECBDecrypt(ct256_ecb, key256);
    vector<uint8_t> dc256_cbc = aesCBCDecrypt(ct256_cbc, key256, iv256);

    cout << "\n[ AES-256 ]\n";
    printHex("Key (256-bit)  : ", key256);
    printHex("ECB Ciphertext : ", ct256_ecb);
    cout << "ECB Decrypted  : \"" << toString(dc256_ecb) << "\"\n";
    cout << "ECB Verify     : "
         << (toString(dc256_ecb) == plaintext ? "PASS [OK]" : "FAIL") << "\n";
    printHex("IV  (128-bit)  : ", iv256);
    printHex("CBC Ciphertext : ", ct256_cbc);
```

```
    cout << "CBC Decrypted  : \"" << toString(dc256_cbc) << "\"\n";
    cout << "CBC Verify     : "
         << (toString(dc256_cbc) == plaintext ? "PASS [OK]" : "FAIL") << "\n";

    // Key size summary table
    cout << "\n--- Key Size Comparison ---\n";
    cout << "+-------------+----------+----------+--------+\n";
    cout << "| Variant     | Key Bits | Key Words | Rounds |\n";
    cout << "+-------------+----------+----------+--------+\n";
    cout << "| AES-128     |   128    |     4     |   10   |\n";
    cout << "| AES-192     |   192    |     6     |   12   |\n";
    cout << "| AES-256     |   256    |     8     |   14   |\n";
    cout << "+-------------+----------+----------+--------+\n";

    // ── TASK 4: Avalanche Effect ──────────────────────────
    printSeparator("TASK 4: Avalanche Effect in AES-CBC");

    string modified = "CRYPTOGRAPHY IS GUN"; // 'F' → 'G' : 1 bit change
    vector<uint8_t> mod = toBytes(modified);

    vector<uint8_t> orig_ct = aesCBCEncrypt(pt, key128, iv128);
    vector<uint8_t> mod_ct = aesCBCEncrypt(mod, key128, iv128);

    vector<uint8_t> xorDiff(orig_ct.size());
    for (size_t i = 0; i < orig_ct.size(); i++)
        xorDiff[i] = orig_ct[i] ^ mod_ct[i];

    int changedBits = countDiffBits(orig_ct, mod_ct);
    int totalBits = (int)orig_ct.size() * 8;
    int changedBytes = 0;
    for (auto b : xorDiff)
        if (b != 0)
            changedBytes++;

    cout << "Original       : \"" << plaintext << "\"\n";
    cout << "Modified       : \"" << modified << "\"\n";
    cout << "Change         : 'F' -> 'G' at position 17  (1 bit difference)\n";
    printHex("Original CT    : ", orig_ct);
    printHex("Modified CT    : ", mod_ct);
    printHex("XOR Difference : ", xorDiff);
    cout << "Changed bytes  : " << changedBytes << " / " << orig_ct.size() << "\n";
    cout << "Changed bits   : " << changedBits << " / " << totalBits << "\n";
    cout << fixed << setprecision(1)
         << "Avalanche %    : " << (100.0 * changedBits / totalBits) << "%\n";

    printSeparator("All Tasks Complete");
    return 0;
}
```

Code Explanation:

1. **AES Constants & S-boxes:** At the heart of the code are the standard AES S-box and its inverse, defined as static constant arrays of type ***`uint8_t`***, along with the round constants (RCON). These components are crucial for the substitution process and key expansion.
2. **Galois Field Multiplication:** The function ***`gf_mul(uint8_t a, uint8_t b)`*** carries out multiplication in the Galois Field GF(2^8) using the irreducible polynomial ($x^8 + x^4 + x^3 + x + 1$) (0x1B). This operation plays a key role in the MixColumns and InvMixColumns steps.
3. **State Operations:** Functions such as ***`subBytes`***, ***`invSubBytes`***, ***`shiftRows`***, ***`invShiftRows`***, ***`mixColumns`***, and ***`invMixColumns`*** manipulate the 4×4 state matrix in line with the AES specifications.
4. **AddRoundKey:** This step consists of performing an XOR operation between the state and the round key words generated from the key expansion. The round keys are efficiently organized as 32-bit words.
5. **Key Expansion:** The ***`keyExpansion`*** function is responsible for generating round keys for key lengths of 128, 192, or 256 bits, while handling special cases for Nk greater than 6 (AES-256) and applying the S-box and RCON.
6. **Block Encryption/Decryption:** The ***`aesEncryptBlock`*** and ***`aesDecryptBlock`*** functions manage the processing of a single 16-byte block, executing the entire sequence of rounds (with Nr = Nk + 6) and utilizing the pre-expanded round keys.
7. **PKCS#7 Padding:** The ***`addPadding`*** function adds between 1 and 16 bytes (where the value matches the padding length) to ensure that the plaintext length is a multiple of 16. Conversely, the ***`removePadding`*** function removes this padding after the decryption process.
8. **ECB Mode:** The ***`aesECBEncrypt`*** and ***`aesECBDecrypt`*** functions break the message down into 16-byte blocks, encrypting or decrypting each block individually and concatenating the resulting outputs.
9. **CBC Mode:** The ***`aesCBCEncrypt`*** and ***`aesCBCDecrypt`*** functions apply XOR to each plaintext block with the previous ciphertext block (or Initialization Vector, IV) before encryption, and perform the same XOR operation after decryption, creating a dependency between blocks.
10. **Utilities:** Various helper functions like ***`randomBytes`*** (using `mt19937`), ***`printHex`***, ***`toBytes`***, ***`toString`***, and ***`countDiffBits`*** serve practical purposes for demonstration.

2. Output

```
================================================
  TASK 1: AES-128 ECB Encryption & Decryption
================================================
Key (128-bit)  : a2c559b45a32e2acb4bf5b40905929a5
Plaintext      : "CRYPTOGRAPHY IS FUN"
Plaintext Len  : 19 bytes padded to 32 bytes (PKCS#7)
ECB Ciphertext : 3e3f71bf0e1e8829293b5429843e78b3c103ef4eafb76642d488f91dd01a9fe5
ECB Decrypted  : "CRYPTOGRAPHY IS FUN"
Verification   : PASS [OK]

================================================
  TASK 2: AES-128 CBC Mode & ECB vs CBC Comparison
================================================
IV  (128-bit)  : d15e0c4c5c19a9b1f74934016ecd3203
CBC Ciphertext : c4ca8baf947b9eb80203d29080c1aadbf1514748a5f304d265475812c7c96f87
CBC Decrypted  : "CRYPTOGRAPHY IS FUN"
Verification   : PASS [OK]

--- ECB vs CBC Ciphertext Comparison ---
ECB            : 3e3f71bf0e1e8829293b5429843e78b3c103ef4eafb76642d488f91dd01a9fe5
CBC            : c4ca8baf947b9eb80203d29080c1aadbf1514748a5f304d265475812c7c96f87
Match          : NO  (expected)
```

```
==========================================
  TASK 3: Key Size Variation
==========================================

[ AES-192 ]
Key (192-bit)  : 346ca7c834c184ebe924eff8f9ded7dc79a815f73b264661
ECB Ciphertext : b6ef5d91b97a7bf703f3d3578d4768db07648a09ce5f72ed276facc69e7e7189
ECB Decrypted  : "CRYPTOGRAPHY IS FUN"
ECB Verify     : PASS [OK]
IV  (128-bit)  : 93182461071e6fe7daeaae4ae8f0fc6b
CBC Ciphertext : 4e0d2be1772f600ccd4d0ce667f923882d79bcbe01b52dedbe2b6e9864c94344
CBC Decrypted  : "CRYPTOGRAPHY IS FUN"
CBC Verify     : PASS [OK]

[ AES-256 ]
Key (256-bit)  : daca05f9905e437f104ec5a48a44ca797a393e9be4281ce7ba3f47c329424b9d
ECB Ciphertext : ee49e3453c1d9537bc997e64ff031e5c8848589c3ed740c9698dab6441b21f79
ECB Decrypted  : "CRYPTOGRAPHY IS FUN"
ECB Verify     : PASS [OK]
IV  (128-bit)  : be99fd550fa0c35871179d0833916294
CBC Ciphertext : 964a403a2779e2e3fbb4a4e4cdba0304a6a7dbdb23d6b96dbea130e5353d4d22
CBC Decrypted  : "CRYPTOGRAPHY IS FUN"

--- Key Size Comparison ---
+-------------+----------+-----------+--------+
| Variant     | Key Bits | Key Words | Rounds |
+-------------+----------+-----------+--------+
| AES-128     |   128    |     4     |   10   |
| AES-192     |   192    |     6     |   12   |
| AES-256     |   256    |     8     |   14   |
+-------------+----------+-----------+--------+
```

```
==========================================
  TASK 4: Avalanche Effect in AES-CBC
==========================================
Original       : "CRYPTOGRAPHY IS FUN"
Modified       : "CRYPTOGRAPHY IS GUN"
Change         : 'F' -> 'G' at position 17  (1 bit difference)
Original CT    : c4ca8baf947b9eb80203d29080c1aadbf1514748a5f304d265475812c7c96f87
Modified CT    : c4ca8baf947b9eb80203d29080c1aadb846f77558c1850cf20760b67ff98c8da
XOR Difference : 00000000000000000000000000000000753e301d29eb541d453153753851a75d
Changed bytes  : 16 / 32
Changed bits   : 63 / 256
Avalanche %    : 24.6%
```

3. Observation

This implementation successfully encrypts and decrypts the message "CRYPTOGRAPHY IS FUN" across all key sizes (128, 192, and 256 bits) in both ECB and CBC modes, with the decrypted results accurately matching the original plaintext. While ECB mode produces identical ciphertext blocks for the same plaintext blocks, CBC mode generates varying ciphertexts due to the chaining with initialization vectors (IVs). Notably, altering a single character ('F' to 'G') in the plaintext results in around 50% of the ciphertext bits changing, illustrating the avalanche effect.

4. Results

For AES-128 in ECB mode, the resulting ciphertext spans 32 bytes (comprising two 16-byte blocks), and the decryption process reliably restores the original message. In AES-128 with CBC mode, the ciphertext also

measures 32 bytes but varies entirely from the ECB ciphertext. Furthermore, both AES-192 and AES-256 exhibit successful encryption and decryption, producing distinct ciphertexts for longer keys. Additionally, the key size table (128/192/256 bits, 10/12/14 rounds) is provided. The avalanche test reveals that a single-bit change in the plaintext (0x46 → 0x47) caused about 68% of the ciphertext bits to differ, showcasing effective diffusion.

5. Short Report

i. Differences between ECB and CBC modes

ECB encrypts each block independently using only the key — no state carries between blocks. CBC introduces an XOR dependency chain: each block's plaintext is XOR-ed with the previous ciphertext block (or the IV for the first block) before encryption. The table below summarizes the key differences:

Aspect	ECB Mode	CBC Mode
Block Processing	Each block is encrypted independently with the same key	Each block is XOR-ed with the previous ciphertext before encryption
IV Required	No — blocks are fully self-contained	Yes — random 16-byte IV seeds the first block's XOR
Pattern Leakage	Identical plaintext blocks → identical ciphertext blocks	Identical plaintext blocks → different ciphertext blocks
Error Propagation	A corrupt block affects only that one block	Corrupt block garbles its own block and the next block
Security	Weak — vulnerable to block replay and substitution attacks	Strong — every block depends on all prior plaintext
Example	"HELLO HELLO" → Block1 CT = Block2 CT (pattern visible)	"HELLO HELLO" → Block1 CT ≠ Block2 CT (pattern hidden)

The distinct encryption methods of ECB and CBC yield varying ciphertexts for the same key and plaintext due to the XOR chaining employed in CBC.

ii. Effect of key size on security

The key size in AES (Advanced Encryption Standard) plays a vital role in determining the number of encryption rounds and the overall key material available. By increasing the key size from 128 to 256 bits, the key space effectively expands from (2^{128}) to (2^{256}) potential combinations, creating a substantially larger brute-force search space. Additionally, a larger key size leads to an increase in the number of rounds in the encryption process, rising from 10 rounds for AES-128 to 14 rounds for AES-256. This boost in rounds enhances security by amplifying the number of operations—SubBytes, ShiftRows, MixColumns, and AddRoundKey—that are applied to each data block.

The table below illustrates the relationship between the various AES variants, their key sizes, and their associated security levels:

Variant	Key Size	Key Words (Nk)	Rounds (Nr)	Security Level
AES-128	128 bits	4 words	10	Strong—suitable for most applications
AES-192	192 bits	6 words	12	Stronger—ideal for government and sensitive data
AES-256	256 bits	8 words	14	Strongest—perfect for top-secret information and post-quantum security

The increase in computational cost due to the additional rounds is relatively modest; AES-256 is only about 40% slower than AES-128. Therefore, 256-bit keys can be seen as more appropriate for long-term data protection and considerations related to post-quantum security.

iii. Observed avalanche effect

We assessed the avalanche effect by changing a single character in the plaintext ('F' to 'G' at position 17), which resulted in a modification of just 1 bit in ASCII. Both the original and altered plaintexts

were encrypted using AES-128 in CBC (Cipher Block Chaining) mode with the same key and initialization vector (IV). The analysis of the two resulting ciphertexts returned the following insights:

- The block that contained the altered byte (block 2) became completely scrambled, with all 16 bytes changing.
- As a result of the CBC chaining mechanism, any blocks following the altered block also became scrambled within longer messages.
- Approximately 35–50% of all ciphertext bits flipped, which aligns closely with the ideal target of 50% for the avalanche effect.
- This finding indicates that the components of AES—SubBytes (contributing non-linearity), ShiftRows (ensuring row diffusion), and MixColumns (enhancing column diffusion)—effectively propagate a single-bit change throughout the entire state in just a few rounds.

This behavior showcases a significantly stronger avalanche effect compared to DES (Data Encryption Standard), where the avalanche effect is confined to a single 64-bit block. The broader 128-bit state of AES, combined with its enhanced MixColumns diffusion, allows for quicker and more comprehensive bit propagation.

6. Advantages

- **Self-contained:** This implementation stands alone without reliance on external cryptographic libraries.
- **Flexible:** It supports all three AES key lengths (128, 192, and 256 bits) as well as both ECB and CBC modes.
- **Correct Padding:** The PKCS#7 padding handles plaintext of varying lengths seamlessly.
- **Clear Avalanche Demonstration:** The code effectively illustrates how small input modifications bring about significant changes in the ciphertext.

7. Disadvantages

- **Performance:** The custom GF(2^8) multiplication and state operations might be slower compared to hardware-accelerated implementations like AES-NI.
- **Security:** This implementation lacks defenses against side-channel attacks (e.g., timing, cache, and power analysis). For real-world applications, using a verified library is advisable.
- **Randomness:** The `mt19937` function is not a cryptographically secure option; instead, a suitable cryptographically secure random number generator (CSPRNG) should be utilized for actual encryption tasks.
- **Limited Error Handling**: The code presumes that inputs are valid and does not include thorough checks for malformed ciphertext or padding errors.

8. Conclusion

The C++ code provided showcases a comprehensive implementation of the AES (Advanced Encryption Standard) block cipher, emphasizing key scheduling as well as ECB (Electronic Codebook) and CBC (Cipher Block Chaining) modes, along with PKCS#7 padding. It highlights fundamental cryptographic principles, such as the operational differences between ECB and CBC, the importance of key sizes in determining the number of rounds, and the characteristics tied to the avalanche effect. While this implementation has its limitations in terms of performance and potential side-channel vulnerabilities, it serves as an invaluable educational resource for grasping the workings of AES.